

Lahore University of Management Sciences
School of Science and Engineering, Department of Electrical Engineering
AI600 – Deep Learning
Spring 2026
Assignment 3

Muhammad Ahmad
25280065

https://github.com/Ahmad-Gill/DeepLearning/tree/main/Assignments/25280065_DeepLearningAssignment3

Analytical

Q1: Forward and Backward Pass

Given

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

$$b = 0$$

$$c = 1$$

$$T = 3$$

$$v = [1 \quad -1 \quad 2 \quad 0.5]$$

Question No. 1 :-

1a)

result after apply filter

$$Z = \begin{bmatrix} 2 & 2 \\ -1 & 3 \end{bmatrix}$$

pre activation -

Apply ReLU is

$$\text{ReLU} \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

DATE: _____

DAY: _____

$$A = \text{Relu}(Z) = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$$

Flatten the feature map

$$A = \begin{bmatrix} 2 \\ 2 \\ 0 \\ 3 \end{bmatrix}$$

Final prediction

$$Y = V \cdot A_{\text{flat}} + C$$

$$Y = \begin{bmatrix} 1 & -1 & 2 & 0.5 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \\ 0 \\ 3 \end{bmatrix} + C(1)$$

$$Y = 1.5 + 1 = 2.5 = Y$$

Loss:-

$$L = \frac{1}{2} (Y - T)^2 = \frac{1}{2} (2.5 - 3)^2$$

$$L = 0.125$$

DATE: _____

DAY: _____

(b)

$$\frac{\partial L}{\partial Y} = \frac{\partial}{\partial Y} \frac{1}{2} (Y - T)^2$$

$$\frac{\partial L}{\partial Y} = Y - T \quad Y = 2.5 \quad T = 3$$

$$\frac{\partial L}{\partial Y} = 2.5 - 3 = -0.5$$

$$\frac{\partial L}{\partial Y} = \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial v_i} \quad \because Y = v_1 a_{11} + v_2 a_{12} + v_3 a_{13} + v_4 a_{14}$$

$$\frac{\partial L}{\partial v_i} = \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial v_i}$$

$$= \frac{\partial L}{\partial Y} \cdot a_{i1}$$

in terms of:

$$\frac{\partial L}{\partial v_i} = \frac{\partial L}{\partial Y} \cdot a_{i1}$$

$$\frac{\partial L}{\partial Y} = (-0.5) \cdot [2, 2, 0, 3]$$

$$= [-1, -1, 0, -1.5]$$

DATE: _____

DAY: _____

$$\frac{\partial L}{\partial a} = (-0.5)(2, 2, 0, 3)$$

$$\frac{\partial L}{\partial a}$$

$$= [-0.5 \quad 0.5 \quad -1 \quad -0.25]$$

$$\frac{\partial L}{\partial A} = \begin{bmatrix} -0.5 & 0.5 \\ -1 & -0.25 \end{bmatrix}$$

$$\frac{\partial L}{\partial z} = -0.5(1, -1, 0, 1)$$

$$\frac{\partial L}{\partial z}$$

$$= \begin{bmatrix} -0.5 & 0.5 \\ 0 & -0.5 \end{bmatrix}$$

$$\frac{\partial L}{\partial z_{11}} = \frac{\partial L}{\partial a_{11}} = \frac{\partial a_{11}}{\partial z_{11}} = (-0.5)(1) = -0.5$$

$$\frac{\partial L}{\partial z_{11}} \quad \frac{\partial L}{\partial a_{11}} \quad \frac{\partial a_{11}}{\partial z_{11}}$$

Conclusion

$$\frac{\partial L}{\partial A} = \begin{bmatrix} -0.5 & 0.5 \\ -1 & -0.25 \end{bmatrix}$$

$$\frac{\partial L}{\partial z} = \begin{bmatrix} -0.5 & 0.5 \\ 0 & -0.25 \end{bmatrix}$$

DATE: _____

DAY: _____

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z_{11}} x_{11} + \frac{\partial L}{\partial z_{12}} x_{12} + \frac{\partial L}{\partial z_{21}} x_{21} + \frac{\partial L}{\partial z_{22}} x_{22}$$

$$= -0.5 \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} + 0.5 \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} - 0.25 \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$\frac{\partial L}{\partial w} = \begin{bmatrix} 0.25 & -1.25 \\ 0.5 & -0.5 \end{bmatrix}$$

(C)

We have

$$\frac{\partial L}{\partial w} = \begin{bmatrix} 0.25 & -1.25 \\ 0.5 & -0.5 \end{bmatrix}$$

The largest absolute gradient is -1.25
in top right weight.

The weight update in the opposite direction
of the gradient.

$$w_{ij} = w_{ij} - \eta \frac{\partial L}{\partial w_{ij}}$$

DATE: _____

DAY: _____

w_{12} the weight update will be the most significant because it has the largest gradient.

w_{12} will change the most by $\eta(1.25)$

So w_{12} will change the most because it has the largest gradient.

w_{12} has the largest impact on the activation because of direct relationship with input patch. a_{12} is positive, so RELU activation is active, means even small change in w_{12} effects the output significantly.

Q3: Pooling and Normalization

DATE: _____

DAY: _____

Question No: 3:-

1a)

We have $A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$

$y = \max(A) = 3$ so, $y = 3$

gradient only flows to the max value of A

gradient w.r.t x :

x_{22} will have non-zero gradient.

other pixels in X (ie x_{11}, x_{12}, x_{21}) will have zero gradients.

Difference from Q1)

- gradient spread across all pixels in X
- Max pooling: Only max value in A affects the gradient; all other pixels in X have zero gradient.

DATE: _____

DAY: _____

$$Y = 3$$

gradient from only to x_{21}
 zero gradients for x_{11}, x_{12}, x_{21}

(b).

$$A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$$

$$\mu = \frac{1}{4} (2 + 2 + 0 + 3) = 1.75$$

$$G^L = \frac{1}{4} \left[(2 - 1.75)^2 + (2 - 1.75)^2 + (3 - 1.75)^2 \right]$$

$$G^L = 1.1875$$

Layer normalization.

$$A_{\text{norm}} = \frac{A - \mu}{\sqrt{G^L + \epsilon}}$$

$$\epsilon = 0$$

$$= \frac{A - \mu}{\sqrt{G^L}} \quad \sqrt{1.1875} = 1.09$$

$$A_{\text{norm}} = \frac{A - \mu}{1.09} = \frac{A - 1.75}{1.09}$$

$$A_{\text{norm}} = \begin{bmatrix} 0.229 & 0.229 \\ -1.607 & 1.149 \end{bmatrix}$$

DATE: _____

DAY: _____

$$\text{mean} = 1.75$$

$$\text{variance} = 1.1875$$

$$A_{\text{norm}} = \begin{bmatrix} 0.229 & 0.229 \\ -1.607 & 1.149 \end{bmatrix}$$

(C)

$$A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix} \quad \text{max}(A) = 3$$

For example if we shift the image to the right by one pixel

$$X_{\text{shifted}} = \begin{bmatrix} 2 & 0 & 2 \\ 0 & 3 & 1 \end{bmatrix} \quad \therefore \text{this is an example.}$$

$$A = \begin{bmatrix} 2 & 0 \\ 3 & 1 \end{bmatrix} \quad \text{max}(A) = 3$$

Before shift = 3

After shift = 3

The max value does not change due to the shift. (as long as the max value is same) This is why max pooling provides Translation invariance. The

Programming: Implementation and Analysis

Task 1: Custom CNNs and Shortcut Learning

Part A: Standard MNIST

1. Architecture Design

The CNN model was designed for the classification of the MNIST dataset. Below is the architecture we used, following the constraints provided:

1.1 Model Architecture

The model consists of the following components:

1. Convolutional Layers:
 - First Convolutional Layer:
 - Input Channels: 1 (grayscale images)
 - Output Channels: 8
 - Kernel Size: 3x3
 - Padding: 1
 - Activation Function: ReLU
 - Pooling: MaxPooling (2x2)
 - Second Convolutional Layer:
 - Input Channels: 8
 - Output Channels: 16
 - Kernel Size: 3x3
 - Padding: 1
 - Activation Function: ReLU
 - Pooling: MaxPooling (2x2)
 - Third Convolutional Layer:
 - Input Channels: 16
 - Output Channels: 32
 - Kernel Size: 3x3
 - Padding: 1
 - Activation Function: ReLU
 - Pooling: MaxPooling (2x2)
2. Fully Connected Layers:
 - First Fully Connected Layer:
 - Input: $32 * 3 * 3$ (flattened feature maps from the third convolutional layer)
 - Output: 128 neurons
 - Activation Function: ReLU
 - Second Fully Connected Layer:
 - Input: 128 neurons
 - Output: 10 neurons (representing the 10 classes for MNIST)

1.2 Parameters

- The total number of trainable parameters is approximately 44,170, which is below the 50,000 limit set for this task.

2. Training and Evaluation

2.1 Training Setup

To train the model, we used the following configuration:

- Optimizer: Adam optimizer was used with a learning rate of 0.001.
- Loss Function: Cross-Entropy Loss was used for multi-class classification (MNIST).
- Learning Rate Scheduler: A StepLR scheduler was applied, which reduces the learning rate by a factor of 0.5 every 5 epochs. This helps to fine-tune the model in the later stages of training.

We also implemented early stopping based on validation loss. If the validation loss did not improve for 5 consecutive epochs, the training was stopped early, and the model from the best epoch (with the lowest validation loss) was restored.

2.2 Epoch-wise Results

The following table shows the training loss, training accuracy, validation loss, and validation accuracy for the first few epochs. These results indicate the progress of the model throughout the training process.

Epoch	Train Loss	Train Accuracy (%)	Validation Loss	Validation Accuracy (%)
1	0.2909	90.78	0.0798	97.47
2	0.0739	97.65	0.0470	98.52
3	0.0533	98.33	0.0429	98.45
4	0.0418	98.71	0.0382	98.74
5	0.0355	98.89	0.0448	98.52

6	0.0295	99.03	0.0406	98.65
7	0.0261	99.15	0.0301	98.93
8	0.0235	99.22	0.0291	99.10
9	0.0207	99.32	0.0351	98.94
10	0.0173	99.44	0.0330	98.97
11	0.0154	99.51	0.0306	98.96
12	0.0148	99.49	0.0296	99.09
13	0.0114	99.61	0.0325	99.02

2.3 Early Stopping

The training process was stopped early at epoch 13 due to no improvement in validation loss after several epochs. Early stopping helped prevent overfitting and ensured that the model was not trained too long, which could have resulted in deteriorating performance on the validation set.

2.3.1 Early Stopping Criteria:

- **Patience:** The model stopped training when the validation loss did not improve for 5 consecutive epochs.
- **Restoring Best Model:** The model with the best validation performance (epoch 13) was restored.

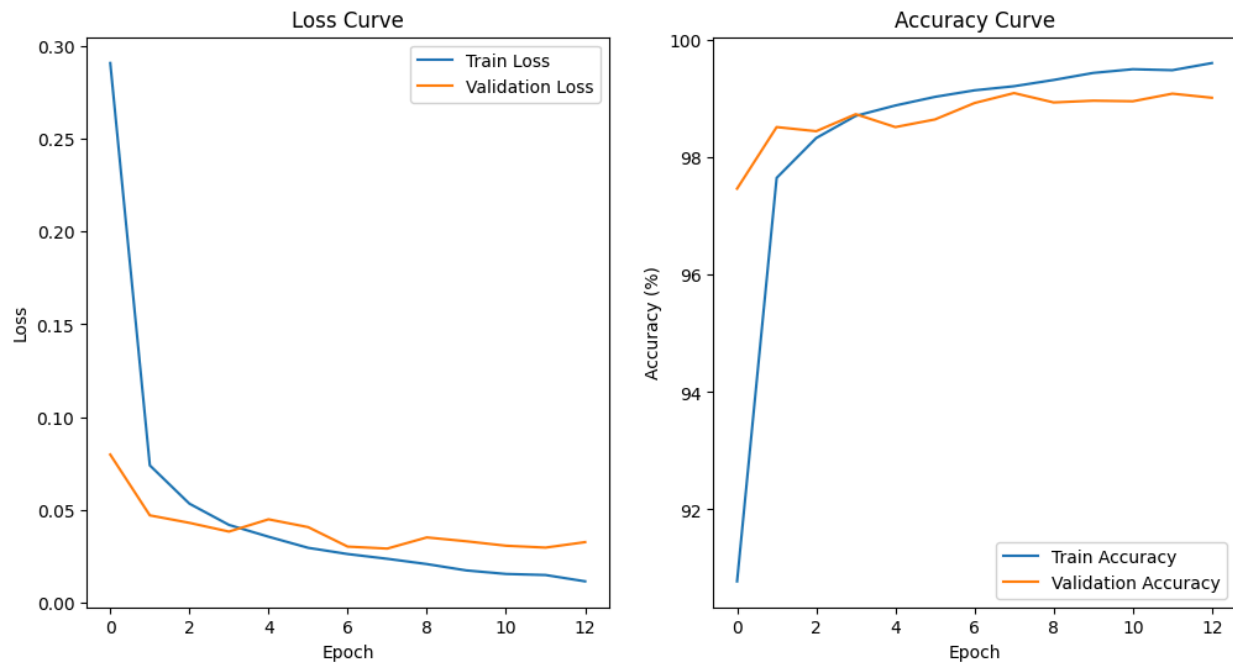
3. Results

3.1 Loss and Accuracy Curves

The following plots show the training loss, validation loss, training accuracy, and validation accuracy over the 13 epochs of training.

3.1.1 Training and Validation Loss Curve

This plot shows how the training loss and validation loss changed over the epochs. Initially, both losses decreased, but after a certain point, the validation loss started fluctuating slightly.



3.2 Test Set Evaluation

After training the model, it was evaluated on the MNIST test set, which contains 10,000 unseen images. The final test accuracy achieved by the model is:

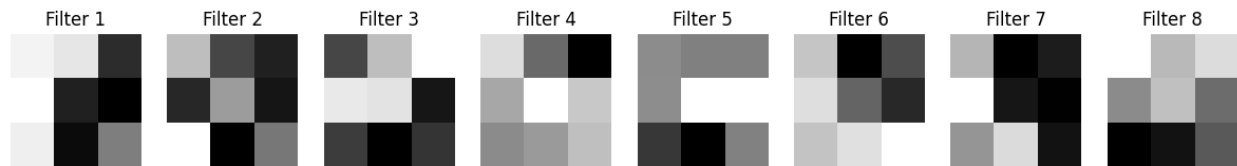
Test Accuracy: 99.18%

This indicates that the model generalizes well and is able to classify unseen data with high accuracy.

Analytical Question 1.1:

Based on the loss curves, the model generalized well. The training and validation losses are close to each other, with no significant gap, and the validation accuracy stabilizes at around 98%. This indicates that the model is performing consistently on both the training and validation sets, without overfitting or underfitting.

Analytical Question 1.2:



The filters from the first convolutional layer are learning to detect basic features like edges and simple textures. Some filters detect horizontal and vertical edges, while others capture patterns like squares and shapes. These basic features help the model identify more complex patterns in later layers.

Part B: Colored-MNIST (C-MNIST)

1. Adaptation

To solve the Colored-MNIST task, the CNN from Part A was adapted to accept **3-channel RGB images** instead of 1-channel grayscale images. This was done by changing the input channels of the first convolutional layer from 1 to 3. The rest of the architecture remained the same.

The adapted model was trained from scratch on the **C-MNIST biased training set** using:

- **Adam optimizer**
- **Cross-Entropy Loss**

Training stopped early at **epoch 14** using early stopping.

2. Training Summary

The model trained smoothly and converged quickly. Both training loss and validation loss decreased consistently during the early epochs, while accuracy improved steadily.

Training Results

Epoch	Train Loss	Train Accuracy	Validation Loss	Validation Accuracy
1	0.2601	93.31%	0.0897	97.35%
2	0.0652	98.09%	0.0470	98.57%
3	0.0430	98.76%	0.0405	98.82%
4	0.0346	98.92%	0.0336	99.02%

5	0.0294	99.10%	0.0281	99.14%
6	0.0241	99.25%	0.0272	99.16%
7	0.0207	99.34%	0.0217	99.40%
8	0.0176	99.47%	0.0196	99.48%
9	0.0146	99.56%	0.0172	99.47%
10	0.0129	99.60%	0.0191	99.43%
11	0.0117	99.64%	0.0187	99.49%
12	0.0100	99.67%	0.0199	99.44%
13	0.0087	99.72%	0.0205	99.44%
14	0.0081	99.74%	0.0213	99.46%

Training was stopped at **epoch 14** because validation loss stopped improving.

3. Evaluation

The trained model was evaluated on two separate test sets.

Test Accuracies

Test Set	Accuracy
Biased Test Set	99.46%
Unbiased Test Set	91.63%

Observation

The model achieved extremely high accuracy on the biased test set and lower accuracy on the unbiased test set. Although the drop is not as dramatic as in some shortcut-learning cases, there is still a clear decrease in performance when the color bias is removed.

4. Analytical Question 1.3

Why does performance drop on the unbiased test set?

The performance drops because the model learns both **shape and color**, but color is strongly correlated with labels in the biased training data. Since color is an easier feature to learn, the model uses it to minimize loss quickly.

Mathematically, the model learns:

$$f(x) = f(\text{shape}, \text{color})$$

instead of relying only on shape. Because color is predictive in training, its weight increases. However, in the unbiased test set, this correlation is removed, making color unreliable and reducing accuracy.

Why did the optimizer prefer color over shape?

The optimizer prefers features that:

- reduce loss quickly
- are easy to learn
- have strong correlation with labels

Color is easier to detect than shape, while shape requires learning complex patterns like edges and strokes. Therefore, the model uses color as a shortcut.

Since the unbiased accuracy is still **91.63%**, the model did learn shape as well, but partial reliance on color caused the drop in performance.

5. Analytical Question 1.4

Strategies to force the model to focus on shape

- **Convert to grayscale** → removes color completely
- **Randomize colors** → breaks color-label correlation
- **Apply color augmentation** → makes color unreliable
- **Train on mixed (biased + unbiased) data** → reduces bias
- **Use regularization (dropout, weight decay)** → avoids shortcuts
- **Adversarial debiasing** → penalizes color-based learning
- **Feature disentanglement** → separates shape and color features

Task 2: Transfer Learning and Interpretability

Part A: Fine-tuning ResNet-18

1. Model Setup

The **ResNet-18** model, pre-trained on **ImageNet**, was loaded from **torchvision.models**. This model was adapted for the **STL-10 dataset** by replacing the final fully connected layer to output **10 classes** instead of the original 1000 classes.

2. Freezing the Backbone

All convolutional layers of ResNet-18 were **frozen** to avoid updating their weights during training, allowing us to focus on fine-tuning the newly added classification head.

3. Classification Head

The final fully connected layer was replaced with a new **linear layer** with **10 outputs** to match the number of classes in the STL-10 dataset.

4. Training

The model was trained on the **STL-10 training set** for **10 epochs**. The **Adam optimizer** was used, and **Cross-Entropy Loss** was the loss function.

Training Loss

- Epoch 1: Loss = 0.9385
- Epoch 2: Loss = 0.3124
- Epoch 3: Loss = 0.2417
- Epoch 4: Loss = 0.1962
- Epoch 5: Loss = 0.1794
- Epoch 6: Loss = 0.1650
- Epoch 7: Loss = 0.1426
- Epoch 8: Loss = 0.1387
- Epoch 9: Loss = 0.1309
- Epoch 10: Loss = 0.1251

5. Evaluation

The model achieved **94.84% accuracy** on the **STL-10 test set**, demonstrating the effectiveness of transfer learning and fine-tuning.

5. Analytical Question 2.1:

Computational Benefits:

- **Reduced Training Time:** Freezing early layers reduces the number of parameters being updated, speeding up training.
- **Fewer Parameters:** Only the final classification layer is trained, making the process more efficient.

Functional Benefits:

- **Transfer of General Features:** Early layers capture low-level features like edges and textures that are useful across tasks.
- **Prevents Overfitting:** Freezing the early layers helps avoid overfitting on the small STL-10 dataset by focusing learning on high-level features.

Features in Early vs. Later Layers:

- **Early Layers:** Learn general features (edges, textures, shapes) that are applicable to various datasets.
- **Later Layers:** Learn task-specific features (object parts, complex patterns) relevant to the target dataset.

Freezing early layers allows the model to use pre-trained knowledge while learning the task-specific features from the new dataset.

Part B: Visualizing Decisions with GradCAM

GradCAM allows us to see which regions of the input image were most important for the model's final prediction by utilizing the gradients flowing into the final convolutional layer. Below is the analysis of the model's predictions using GradCAM visualizations.

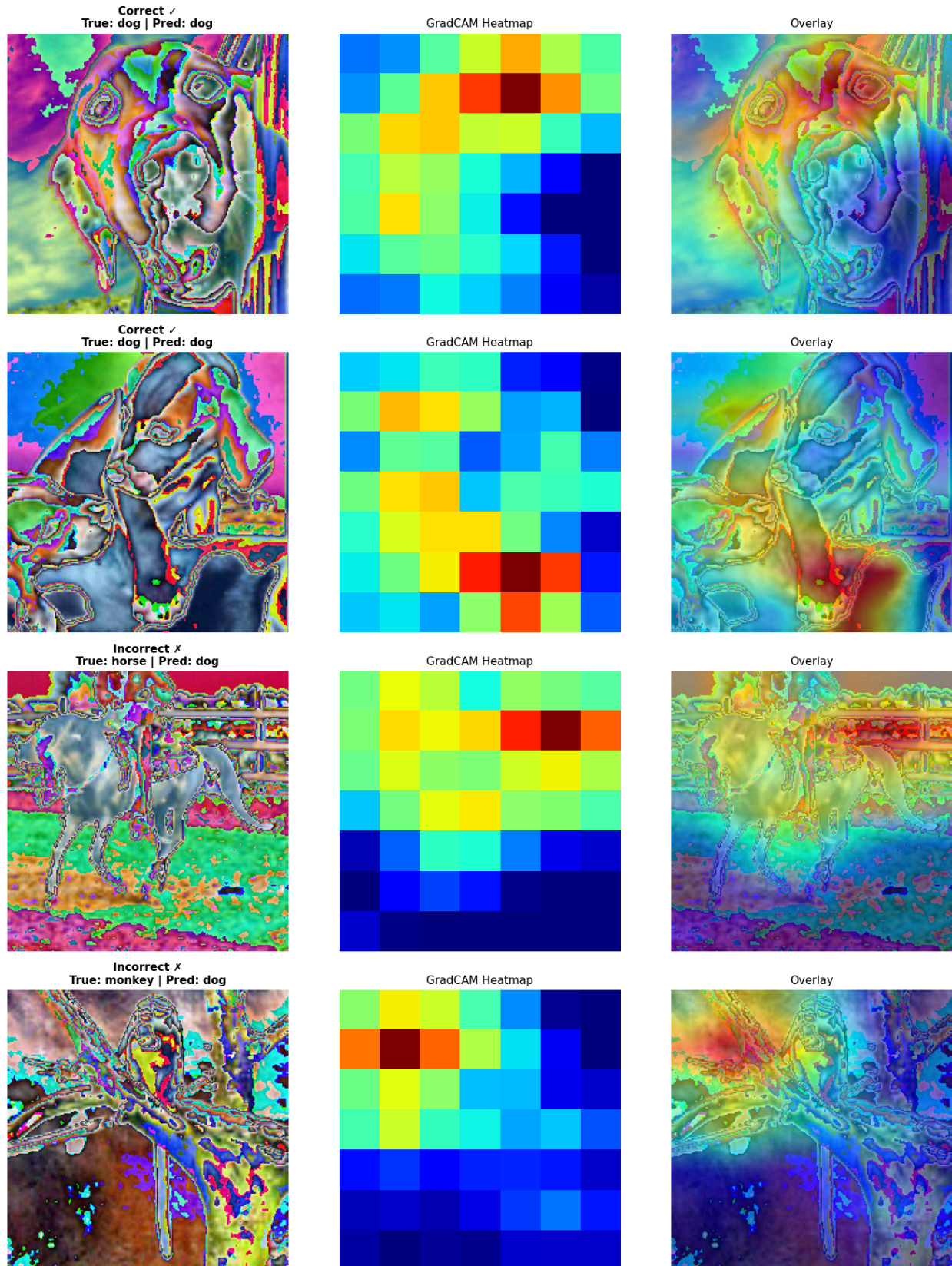
1. Implementation:

- GradCAM was implemented using the pytorch-grad-cam library, which extracts activation maps from the final convolutional layer (layer4) of the fine-tuned ResNet-18 model.

2. Visualization:

- Four images from the STL-10 test set were selected:
 - Two images that the model classified correctly.
 - Two images that the model classified incorrectly.
- For these images, GradCAM heatmaps were generated and overlaid on the original images to visualize which regions of the image were the most influential in making the prediction.

Part B — GradCAM Visualizations (ResNet-18, STL-10)



3. **Analytical Question 2.2:**

○ **Correct Predictions:**

- **First Image** (Pred: dog, True: dog): The heatmap highlights the dog's face and body as the main focus, showing that the model is indeed focusing on the primary object (the dog) and not the background.
- **Second Image** (Pred: dog, True: dog): The heatmap focuses on the horse's body, which is likely a misinterpretation, but it does focus on an object rather than the background.

4. **Analytical Question 2.3:**

○ **Incorrect Predictions:**

- **First Image** (Pred: dog, True: horse): The heatmap focuses on the horse, but the model mistakenly classifies the horse as a dog. The model might have been distracted by the horse's shape but not focused on the correct features.
- **Second Image** (Pred: dog, True: monkey): The heatmap shows a focus on the monkey's face and background. The model likely made the mistake because it misinterpreted the background context (likely the shape of the monkey) instead of correctly identifying the object.